Foundations of Operations Research - Notes - v0.6.0

260236

October 2026

Preface

Every theory section in these notes has been taken from the sources:

- Course slides. [1]

About:

GitHub repository

These notes are an unofficial resource and shouldn't replace the course material or any other book on foundations of operations research. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

Contents

1 Introduction **5**
1.1 What is Operations Research? 5
1.2 Decision-Making Problems 9
1.3 The Scheme of an Operations Research Study 15
1.4 Historical Origins of Operations Research 21
1.5 Building Mathematical Optimization Models 23

Index **39**

Algorithms

1 Introduction

1.1 What is Operations Research?

Let's start with a definition of Operations Research (OR).

Definition 1: Operations Research (OR)

Operations Research (OR) is a branch of mathematics that uses **mathematical models** and **quantitative methods** to analyze complex decision-making problems and find optimal or near-optimal solutions.

So Operations Research helps us make better decisions. Suppose we have a real-world problem such as:

- *Which machine should execute each job?*
- *How should a company allocate limited resources?*
- *What is the shortest route from A to B?*
- *How many workers should be scheduled?*
- *Where should a facility be built?*

In all these cases, there are many possible decisions. Some are infeasible, some are feasible but bad, and some are better than others. OR tries to make this decision process **systematic and quantitative**.

In general, the idea is simple:

1. Real-world problem.
2. Formulate a mathematical representation.
3. Analysis.
4. Decision-making to find the best decision.

Example 1: Manufacturing Decision Problem

Imagine a company deciding how much of three products to manufacture. The real-world description may contain things like: limited workforce, limited machine hours, profit per product, production times.

OR translates those facts into mathematics. We might introduce variables such as:

$$x_1 = \text{number of units of product 1},$$
$$x_2 = \text{number of units of product 2},$$

And then construct an objective such as:

$$\max\ 5x_1 + 8x_2$$

Subject to resource constraints.

From the above example, we can see that OR is not simply "*doing calculations*". A large pat of OR is deciding **how to represent the decision problem mathematically in the first place**. To study decision problems, we have three major quantitative approaches:

- **Optimization** (*what should we do?*): we have decision variables, constraints, and an objective function, and we want the best feasible solution:

  $$\min \text{ or } \max \ f(x) \quad \text{s.t. } x \in X$$

  For example, minimize transportation cost while satisfying demand.

- **Game Theory** (*what should we do when others are also making strategic decisions?*): we have **multiple decision-makers** whose choices interact. The outcome for one player depends on what the others do. For example, two competing companies choosing prices.

- **Simulation** (*what will probably happen if we do this?*): we build a model of how a system behaves and then experiment with it. For example, simulate customers arriving at a call center to estimate waiting times under different staffing levels.

In these notes, we will focus on **optimization**.

⚖ Optimal vs Near-Optimal solutions

In the previous definition, we mentioned that OR finds **optimal or near-optimal solutions**. So the natural question is: "*why not always optimal?*". If we have an optimization problem:

$$\min_{x \in X} f(x)$$

An **optimal solution** x^* satisfies:

$$f(x^*) \leq f(x) \qquad \forall x \in X$$

It is literally the best feasible solution. But some real optimization problems are computationally extremely difficult. If the set of alternatives is enormous, finding the true optimum may **take too much time**. Then we may instead accept a solution that is:

- Feasible;
- Computationally obtainable;
- Very close to the best known solution.

That is a **near-optimal solution**. Later in the notes this distinction will become much more important, especially when we discuss combinatorial and integer optimization.

OR as an interdisciplinary field

Operations Research sits at the intersection of several areas: Applied Mathematics, Computer Science, Economics, and Industrial Engineering. Each contributes something different:

- **Applied mathematics**. Mathematics gives us the language used to describe the problem. For example:

  $$\min f(x) \quad \text{s.t. } g_i(x) \leq 0$$

  It also gives us the theoretical tools to determine whether solutions exist, whether they are optimal, and what properties they have.

- **Computer science**. A mathematical model is not enough. Suppose a problem has 2^{100} possible solutions. Writing the problem mathematically is easy; checking all solutions is impossible in practice. We therefore need algorithms to find good solutions efficiently.

  This is where computer science enters:

  $$\text{Model} \longrightarrow \text{Algorithm} \longrightarrow \text{Computer implementation}$$

  These notes will repeatedly follow exactly this pattern. For example, later we will see shortest-path algorithms, minimum spanning-tree algorithms, maximum-flow algorithms, Simplex, and Branch-and-Bound.

- **Economics**. Many decision problems involve quantities such as: profit, cost, scarce resources, prices, trade-offs. So economic reasoning often determines what the objective function should represent. For instance:

  $$\max \text{ profit} \quad \text{or} \quad \min \text{ operating cost}$$

  Later, LP duality will even give an explicit economic interpretation to certain mathematical quantities.

- **Industrial Engineering**. Industrial engineering provides many of the real problems that OR was designed to solve: scheduling, logistics, transportation, production planning, inventory, facility location, and supply chains.

So OR is not mathematics developed in isolation. It is **mathematics designed around real decision problems**.

> **Key Takeaways 1**
>
> - **Operations Research (OR)** uses mathematical models and quantitative methods to support complex decision-making and find **optimal or near-optimal solutions**.
> - Its overall goal is to **help make better decisions** by translating a real-world problem into a mathematical model that can be analyzed and solved.

- The main quantitative approaches introduced in OR are:
  - **Optimization**: find the best feasible decision;
  - **Game Theory**: study interacting decisions of multiple decision-makers;
  - **Simulation**: evaluate how a system behaves under alternative decisions.
- A **near-optimal solution** may be accepted when finding the true optimum is computationally too expensive.
- OR is interdisciplinary, combining **applied mathematics, computer science, economics, and industrial engineering**.

1.2 Decision-Making Problems

A **Decision-Making Problem** means choosing one feasible alternative among many, according to one or more criteria. It contains three main ingredients: alternatives, feasibility conditions, and criterion.

Suppose we have a set of possible decisions S. Not every decision may be allowed, so only some belong to the **feasible set** $X \subseteq S$. Then we judge feasible solutions according to some criterion (e.g., minimize cost, minimize time, maximize profit, etc.). So conceptually:

$$\text{Choose } x \in X \text{ so that } x \text{ is as good as possible}$$

The **difficulty** is often that there are **too many alternatives**. That is why we need models and algorithms instead of trying possibilities one by one (brute force).

Assignment Problem

Suppose there are m **jobs**:

$$J_1, \ldots, J_m$$

And m **machines**:

$$M_1, \ldots, M_m$$

Every job can be executed by every machine. Let t_{ij} be the **execution time** of job J_i on machine M_j. For example:

	M_1	M_2	M_3
J_1	2	6	3
J_2	8	4	9
J_3	5	7	8

The decision is "*which job should be assigned to which machine?*". But there are **constraints**:

- Every job must be assigned to **exactly one machine**;
- Every machine must be assigned to **exactly one job**.

And the **objective** is to **minimize the total execution time**.

Example 2: $m = 3$

For $m = 3$, one possible assignment is:

$$J_1 \to M_1, \quad J_2 \to M_2, \quad J_3 \to M_3$$

With total time:

$$t_{11} + t_{22} + t_{33} = 2 + 4 + 8 = 14$$

Another possible assignment is:

$$J_1 \to M_3, \quad J_2 \to M_2, \quad J_3 \to M_1$$

With total time:

$$t_{13} + t_{22} + t_{31} = 3 + 4 + 5 = 12$$

So the second one is better.

Why is this an OR problem? Because the number of feasible assignments grows very quickly. With m jobs and m machines, there are $m!$ possible assignments exist. For example, with only $m = 10$ jobs and machines, there are already:

$$10! = 3'628'800 \text{ possible assignments}$$

So even a fairly small instance already has millions of possibilities. Therefore, a compactly described problem can have an enormous solution space. That theme will appear everywhere in OR.

Network design problem

We represent the system with a graph

$$G = (N, E)$$

Where:

- N is the set of nodes;
- E is the set of possible links.

Each edge:

$$\{i, j\} \in E$$

Has a cost c_{ij}. The decision is: ***which links should we build?*** The **requirements** are:

- All nodes must ultimately be connected;
- Total cost should be as small as possible.

So we want to select a subset $E' \subseteq E$, such that the resulting network remains connected and:

$$\sum_{\{i,j\} \in E'} c_{ij}$$

Is minimized. This is the problem that will later lead to the Minimum Spanning Tree.

But, why is it difficult? If there are $m = |E|$ possible edges, each edge can conceptually be: *selected* or *not selected*. Therefore there are up to 2^m possible subsets of edges. For $m = 50$:

$$2^{50} \approx 1.13 \times 10^{15}$$

Of course, many of these subsets are infeasible because they do not connect all nodes. But brute-force enumeration is still hopeless. So, again, we have a **huge search space**, and we **need algorithms to find good solutions**.

Shortest path problem

We again use a graph, but the **decision is different**. Suppose the graph represents a road network. **Nodes** represent **locations**, and each **arc** has a **quantity** such as: distance, travel time, monetary cost. Given an origin s and destination t, the problem is to find the minimum-cost path from s to t.

For example, if we have two possible paths:

1. $s \to a \to t$
2. $s \to b \to c \to t$

We compute their costs by summing the arc costs. If:

$$c(s, a) + c(a, t) = 12$$

While:

$$c(s, b) + c(b, c) + c(c, t) = 8$$

The second path is preferable.

The important point is that the criterion need not literally be distance. A "*shortest*" path can mean: distance, travel time, or monetary cost. So mathematically, "*shortest*" really means "**minimum total arc weight**" according to whatever quantity the weights represent.

Already notice that all three problems fit the same OR pattern:

- **Assignment**
  - **Decision**: job-machine pairs
  - **Feasibility**: one job per machine and vice versa
  - **Objective**: minimize total execution time
- **Network design**
  - **Decision**: which edges to select
  - **Feasibility**: all nodes connected
  - **Objective**: minimize total edge cost
- **Shortest path**
  - **Decision**: which path to follow
  - **Feasibility**: valid path from source to destination
  - **Objective**: minimize path cost

That abstraction is extremely important. At first the applications look unrelated, but OR looks underneath the story and asks: "*what are the decisions? What makes them feasible? What do we optimize?*".

Example 3: Personnel scheduling

Suppose a hospital must prepare a weekly work schedule. The decision concerns things such as:

- Who works;
- On which day;
- During which shift.

The hospital has staffing requirements. For example, "*at least 5 nurses on Monday morning*". It may also have constraints such as:

- Maximum weekly hours;
- Rest periods;
- Employee availability.

The objective might be to minimize the number of employees required, while still satisfying all staffing requirements.

Again:

- **Decision**: schedule of employees;
- **Constraints**: coverage and employment rules;
- **Objective**: use as few resources as possible.

This kind of problem becomes a classic scheduling/integer optimization problem.

Example 4: Service management

Imagine a bank, airport, supermarket, or public office. At each moment, management must decide "*how many service counters should be open?*".

Opening more counters improves service but costs more. Opening too few counters reduces staffing cost but increases waiting time. We might impose a service requirement such as:

$$\text{average waiting time} \leq 5 \text{ minutes}$$

And try to minimize the number of open desks. Usually, the trade-off is between lower operating cost and better service quality.

This example is slightly different from the earlier ones because quantities such as customer arrivals and waiting times may involve **uncertainty**. That will later be one reason why simulation can be useful.

Multicriteria decisions

So far, most examples had one clear objective: minimize cost, minimize time, or maximize profit. But many **real problems have several objectives**.

For instance, consider choosing a laptop. We might care about price, weight, performance. The difficulty is that these objectives can conflict.

- Laptop A may be cheaper, heavier and slower.
- Laptop B may be more expensive, lighter and faster.

Then there may be no universally "*best*" laptop. The decision depends on how much we value each criterion. This is called a **multicriteria** or **multi-objective problem**.

Conceptually, instead of one **objective function** $f(x)$, we may have **several**:

$$f_1(x), f_2(x), \ldots, f_k(x)$$

For example:

$$\min f_1(x) = \text{price}, \quad \min f_2(x) = \text{weight}, \quad \max f_3(x) = \text{performance}$$

Later we will see ways of reducing this to a single optimization model.

Example 5: Maximum clique as a decision problem

This is another graph problem, but unlike shortest path or spanning tree, it is **computationally much harder**.

Suppose we have a social network represented by

$$G = (N, E)$$

Each node is a person. An edge means two people are connected. A **Clique** is a **subset of nodes in which every pair is connected**. For example, if:

$$\{A, B, C, D\}$$

Is a clique, then every pair must have an edge:

$$\{A, B\}, \{A, C\}, \{A, D\}, \{B, C\}, \{B, D\}, \{C, D\}$$

So a clique represents a group where everyone is directly connected to everyone else. The **Maximum Clique Problem** is to find a clique containing the largest possible number of nodes. The objective is therefore:

$$\max |\text{clique}|$$

One application is identifying highly interconnected communities in social networks.

Why is this example especially interesting? Because the problem looks simple to state, but it is algorithmically difficult. And this is

an important OR theme: some problems are easy to describe but hard to solve. Shortest path and maximum clique are both graph problems, but their computational difficulty is radically different. Later, when complexity enters the course, maximum clique becomes a classic example of a hard combinatorial optimization problem.

Key Takeaways 2

- A **decision-making problem** consists in choosing a feasible solution among many alternatives according to one or more criteria. Its fundamental ingredients are:

  $$\boxed{\text{Decisions} + \text{Feasibility conditions} + \text{Objective/criteria}}$$

- Different real-world problems share the same OR structure:
  - **Assignment**: assign jobs to machines minimizing total execution time;
  - **Network design**: select links that connect all nodes at minimum total cost;
  - **Shortest path**: find an s-t path of minimum total arc weight.
- The number of alternatives can grow extremely quickly: an assignment problem with m jobs has $m!$ feasible assignments, while a network with m possible edges has up to 2^m subsets of edges. Hence, **enumerating all alternatives is generally not a viable solution method**.
- A criterion does not have to represent only monetary cost: it may represent **time, distance, profit, service quality**, or several conflicting quantities in a **multicriteria problem**.
- A **problem** can be **easy to state but difficult to solve**. The shortest-path and maximum-clique problems are both graph problems, yet their computational difficulty is very different.

1.3 The Scheme of an Operations Research Study

A standard workflow of an Operations Research project is as follows:

Problem → Model → Algorithm → Implementation → Analysis of results

The important point is that OR is not just "*solve a formula*". Most of the intellectual work often happens **before** the algorithm is run. Let us go through the steps one by one.

1. **Step 1: Define the real problem**. We start from a real decision situation. For example: "*a company must decide how many units of each product to manufacture*". At this stage, the problem is usually described in natural language. We need to identify things such as:
   - What decisions must be made;
   - What the objective is;
   - What resources are limited;
   - What requirements must be respected;
   - What data are available.

   This step is crucial because if we misunderstand the real problem, then even a perfectly solved mathematical model may be useless. So the first question is: "***What exactly are we trying to decide?***".

2. **Step 2: Build the mathematical model**. Now we translate the real problem into mathematics. Typically we identify: **decision variables**, an **objective function**, and **constraints**.

   For example, suppose:

   $$x_1 = \text{number of units of product 1}, \quad x_2 = \text{number of units of product 2}$$

   If profits are 5 and 8, we may write:

   $$\max 5x_1 + 8x_2$$

   If total available production time is 100, we might have:

   $$2x_1 + 4x_2 \leq 100$$

   The real problem has now become an **optimization model**. So:

   $$\text{natural language} \rightarrow \text{mathematical language}$$

   This modeling step is one of the main skills the FOR course aims to teach.

3. **Step 3: Select or develop an algorithm**. Once the mathematical model is known, we ask: "*What kind of optimization problem is this?*" The answer determines which algorithm is appropriate. For example:

   $$\text{Shortest path} \rightarrow \text{shortest-path algorithm}$$

   $$\text{Minimum spanning tree} \rightarrow \text{Prim/Kruskal-type algorithm}$$

Linear Programming → Simplex

Integer Linear Programming → Branch-and-Bound

So the model acts as a bridge between the application and the algorithm. The same algorithm can solve many completely different real-world problems if they have the same mathematical structure. That is one of the major strengths of OR. For example:

road routing and packet routing in a network

May both reduce to a shortest-path problem. The physical interpretation is different, but mathematically the structure is the same.

4. **Step 4: Implement or use software**. For realistic instances, solving the model manually is generally impossible. Therefore we implement the algorithm or use an existing solver. In this course, this will often mean something like:

   Mathematical model → Python representation → CPLEX

   The solver computes a numerical solution. But this does **not** mean the human work is finished. A solver only solves the model that we give it. If the model is wrong, then correct optimization of the wrong model is still wrong. This is an important practical principle.

5. **Step 5: Analyze the results**. The solver returns a solution to the mathematical model. But we must still interpret the solution in terms of the real problem. For example, if the solver returns:

   $$x_1 = 40, \qquad x_2 = 10$$

   We must ask:

   - *What does this mean in the real problem?*
   - *Is the solution realistic?*
   - *Are the assumptions reasonable?*
   - *Is it implementable?*
   - *Does it behave sensibly if parameters change?*

   So the output must be translated back:

   mathematical solution → real decision

Mathematical models as simplified representations

A mathematical model is **not the real system itself**. It is a simplified representation of the aspects of reality that matter for the decision. This distinction is fundamental.

Suppose we want to optimize delivery routes. The real system contains enormous detail: weather, traffic, driver behavior, vehicle reliability, road closures,

customer preferences, fuel prices, parking availability, etc. A useful model cannot necessarily include everything. Instead, we might retain only locations, distances, travel times, vehicle capacities, and delivery demands. So we perform an abstraction:

$$\text{Complex reality} \longrightarrow \text{Relevant mathematical structure}$$

The goal is therefore **not** to construct the most detailed model possible. The **goal** is to construct a **model** that is sufficiently realistic to capture the essential features of the real problem, but sufficiently simple to be solved. There is **always a trade-off**.

What must a model capture? To create a useful model, we identify two things:

- **Fundamental elements**. These are the objects involved in the decision. For a production problem, they might be products, resources, production quantities, and profits. For a routing problem, they might be nodes, roads, and distances. For scheduling, they might be employees, shifts, and demand requirements.
- **Relationship between the elements**. The model must also represent how these quantities interact. For example, production consumes resources, and assigning a worker to a shift contributes to demand coverage.

  Mathematically, these relationships typically become **constraints**. For example:

  $$2x_1 + 3x_2 \leq 100$$

  May represent the relation between production decisions and available machine time. So constraints are not arbitrary inequalities. They encode real-world relationships.

Modeling necessarily involves assumptions. Suppose we write:

$$\text{travel time on road } (i, j) = 10 \text{ minutes}$$

In reality, travel time varies. The **model may therefore implicitly assume** that travel times are known and fixed because that simplification allows us to solve the problem using deterministic optimization.

Similarly, the production example may assume that everything produced can be sold. This eliminates uncertain demand from the mod

Assumptions are therefore necessary. The important question is not "*does the model perfectly reproduce reality?*", because **no model does**. The important question is: "***are the simplifications acceptable for the decision we want to make?***".

Feedback and iterative refinement of the model

The OR process is not normally:

$$A \to B \to C \to D$$

With no return. Instead it is **iterative**. A better representation is:

$$\text{Problem} \rightleftarrows \text{Model} \rightleftarrows \text{Algorithm} \rightleftarrows \text{Implementation} \rightleftarrows \text{Results}$$

At any stage, we may discover something that forces us to reconsider a previous stage.

Example 6: Feedback

Imagine we construct a staff-scheduling model and obtain the mathematically optimal solution. The solver says:

$$\text{Employee A works 16 consecutive hours.}$$

Perhaps the original model forgot a labor-rule constraint. Then the optimization algorithm is not the problem. The **model is incomplete**. We go back and add, for instance:

$$\text{maximum consecutive working hours} \leq 8$$

Then we solve again. So:

$$\text{results} \to \text{discover missing constraint} \to \text{modify model} \to \text{solve again}$$

Example 7: The model is computationally too hard

Suppose we model the real system extremely accurately. But then solving the **model requires days of computation**. We might decide to simplify it, thus:

$$\text{Algorithm/implementation difficulty} \to \text{model redesign}$$

This illustrates an important OR principle:

$$\text{model quality and computational tractability must be balanced}$$

A perfect model that cannot be solved may be less useful than a slightly simplified model that can provide a good decision quickly.

Validation. A central part of this feedback loop is **validation**. After obtaining a solution, we ask whether the model behaves consistently with reality.

For example:

- *Does the solution make operational sense?*
- *Does it violate an implicit requirement?*
- *Are the model parameters accurate?*
- *Do small changes in data create absurdly large changes in the solution?*

If problems appear, the model is refined. So an OR study often looks like:

1. Understand the problem
2. Model it
3. Solve it
4. Interpret and validate
5. Refine if necessary
6. Repeat

In summary, there are 3 different levels that we should not confuse:

$$\text{Problem} \neq \text{Model} \neq \text{Algorithm}$$

Where:

- The **problem** is the real decision situation.
- The **model** is its mathematical representation.
- The **algorithm** is the method used to solve that mathematical representation.

For example:

- "*Find the cheapest road network*" is the real problem.
- "*Minimum spanning tree problem on* $G = (N, E)$" is the mathematical model.
- "*Kruskal's algorithm*" is one possible solution method.

Key Takeaways 3

- **A mathematical model is a simplification of reality**, not a complete reproduction of the real system.
- A good model keeps the **fundamental elements** of the decision problem and the **relationships** among them.
- Real-world relationships are translated into mathematical structures, especially **constraints**.
- Every model relies on **assumptions**; the important question is whether those assumptions are reasonable for the decision being

made.

- There is always a trade-off between **realism** and **solvability**:

  $$\boxed{\text{model accuracy} \leftrightarrow \text{computational tractability}}$$

- An OR study is **iterative**, not a one-way pipeline. Results may reveal missing constraints, wrong assumptions, inaccurate parameters, or excessive computational complexity.
- **Validation** checks whether the obtained solution is meaningful and consistent with the real system.
- The model should be **refined and solved again** whenever validation reveals problems.
- Always distinguish:

  $$\boxed{\text{Problem} \neq \text{Model} \neq \text{Algorithm}}$$

  Where the **problem** is the real decision situation, the **model** is its mathematical representation, and the **algorithm** is the method used to solve the model.

Thus, OR transforms a real decision problem into a solvable mathematical model, validates the result, and iteratively refines the model when necessary.

1.4 Historical Origins of Operations Research

This section is short and mostly historical, but it explains **why OR emerged exactly when it did**.

✈ OR during World War II

Operations Research developed as a recognizable discipline during **World War II**. Military organizations faced complex operational problems involving scarce resources, uncertainty, and many possible decisions. Teams of scientists were asked to study how operations could be conducted more efficiently.

Typical problems included:

- Where to place radar systems;
- How to organize convoys;
- How to allocate limited military resources;
- How to improve logistics.

The basic idea was already the same as modern OR:

$$\text{Real operational problem} \rightarrow \text{quantitative analysis} \rightarrow \text{better decision}$$

The word **operations** in *Operations Research* comes from this origin. Researchers were literally studying military operations. The important conceptual point is that OR was born from a need to make **better use of scarce resources under complex conditions**.

🏭 Expansion to industry and management

After the war, the techniques developed for military applications became available for civilian use. At the same time, companies and organizations were becoming much larger and more complex.

This created problems such as:

- Production planning;
- Transportation;
- Inventory management;
- Workforce scheduling;
- Logistics;
- Supply-chain coordination.

So methods originally designed for military operations naturally transferred to industrial and managerial problems. The same underlying question remained: ***how should limited resources be allocated to achieve the best result?*** For example:

$$\text{military vehicles} \longrightarrow \text{delivery vehicles}$$

$$\text{troop deployment} \longrightarrow \text{workforce scheduling}$$

$$\text{military logistics} \longrightarrow \text{supply-chain logistics}$$

The applications changed, but the mathematical structure was often similar. This is why OR became closely associated with **Management Science**. The two terms are often used almost interchangeably, although *Management Science* tends to emphasize applications to managerial decision-making.

Role of algorithms and computing

OR could expand rapidly because two technological developments happened at the same time. First, there was strong progress in **optimization methods and numerical algorithms**. Second, **computers became increasingly available**. These two developments reinforce each other. A mathematical optimization model may have an enormous number of possible solutions.

For example, with n binary decisions:

$$x_i \in \{0, 1\}$$

There are potentially 2^n different combinations. For $n = 100$:

$$2^{100} \approx 1.27 \times 10^{30}$$

So just having a mathematical formulation is not enough. We need:

$$\text{Efficient algorithms} + \text{Computing power}$$

Without efficient algorithms, computers would simply enumerate possibilities too slowly. Without computers, even good algorithms would be limited to very small problems. So modern OR emerged from the combination:

$$\text{Mathematical modeling} + \text{Algorithmic advances} + \text{Computers}$$

1.5 Building Mathematical Optimization Models

The **goal** of this section is to learn how to go from a verbal description of a decision problem to a mathematical model. The recurring structure is:

$$\text{Decision variables} + \text{Objective function} + \text{Constraints} + \text{Variable domains}$$

This is the basic language of mathematical optimization. The following examples illustrate how to use it.

The basic ingredients of an optimization model

Suppose a problem asks:

"*What should we decide so that some objective is optimized while respecting certain restrictions?*"

We build the model in four steps.

1. **Decision variables**. These represent the decisions under our control. For example:
$$x_j = \text{number of units of product } j$$
Or:
$$x_j = \begin{cases} 1 & \text{if investment } j \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$
The first modeling question should almost always be: "***what do we need to decide?***". The answer usually tells us what our variables should be.

2. **Objective function**. The objective tells us what "best" means. Typical forms are:
$$\max \text{ profit}$$
$$\min \text{ cost}$$
$$\min \text{ time}$$
$$\max \text{ return}$$
Mathematically:
$$\max f(x) \qquad \text{or} \qquad \min f(x)$$
The objective is therefore a numerical measure of the quality of a feasible decision.

3. **Constraints**. Constraints represent the requirements that a valid solution must satisfy. Examples include:
$$\text{resource usage} \leq \text{available resource}$$
$$\text{number of selected items} \leq 5$$
Or:
$$\text{distance from forbidden point} \geq 100$$
Constraints determine which candidate solutions are **feasible**.

4. **Variable domains**. We must also specify what values the variables are allowed to take. For example:
   - Continuous: $x_j \in \mathbb{R}$ or $x_j \geq 0$.
   - Integer: $x_j \in \mathbb{Z}$.
   - Binary: $x_j \in \{0, 1\}$.

   This is not a minor technical detail. The domain often completely changes the nature and difficulty of the optimization problem.

So a model can be summarized schematically as:

$$\begin{aligned} \max / \min \quad & f(x) \\ \text{s.t.} \quad & \text{constraints} \\ & x \in \text{appropriate domain} \end{aligned}$$

Now let us see this structure in actual examples.

Example 8: Production planning (continuous decision variables)

A company produces three electronic devices:

$$D_1, \quad D_2, \quad D_3$$

Every product goes through three phases:

1. Assembly;
2. Refinement;
3. Quality control.

The required processing times are:

	D_1	D_2	D_3
Assembly	80	70	120
Refinement	70	90	20
Quality control	40	30	20

All values are in minutes per unit. The available capacities are:

Phase	Available minutes
Assembly	30′000
Refinement	25′000
Quality control	18′000

The unit profits are:

	D_1	D_2	D_3
Profit (k€)	1.6	1	2

We assume everything produced can be sold. The goal is: choose production quantities to maximize total profit.

Step 1: Decision variables. *What must the company decide?* How many units of each device to produce. Therefore define:

$$x_1 = \text{quantity of } D_1$$

$$x_2 = \text{quantity of } D_2$$

$$x_3 = \text{quantity of } D_3$$

Compactly:

$$x_j = \text{number of units of } D_j$$

Step 2: Objective function. Each unit of D_1 gives 1.6 k€. So producing x_1 units gives:

$$1.6x_1$$

Likewise:

$$D_2: \quad 1x_2 = x_2$$

$$D_3: \quad 2x_3$$

Therefore **total profit** is:

$$1.6x_1 + x_2 + 2x_3$$

We want to maximize it:

$$\boxed{\max z = 1.6x_1 + x_2 + 2x_3}$$

Notice the modeling pattern:

$$\text{unit contribution} \times \text{quantity}$$

This appears constantly in OR.

Step 3: Assembly capacity. One unit of D_1 needs 80 assembly minutes. Therefore x_1 units need:

$$80x_1$$

Similarly:

$$70x_2$$

For D_2, and:

$$120x_3$$

For D_3. Hence **total assembly usage** is:

$$80x_1 + 70x_2 + 120x_3$$

But only 30′000 minutes are available:

$$\boxed{80x_1 + 70x_2 + 120x_3 \leq 30000}$$

The logic is:

$$\text{resource consumption} \leq \text{resource availability}$$

This is one of the most common constraint templates in optimization.

Step 4: Refinement capacity. Exactly the same reasoning gives:

$$\boxed{70x_1 + 90x_2 + 20x_3 \leq 25000}$$

Step 5: Quality-control capacity. Likewise:

$$\boxed{40x_1 + 30x_2 + 20x_3 \leq 18000}$$

Step 6: Variable domains. Obviously, we cannot produce a negative quantity:

$$x_1, x_2, x_3 \geq 0$$

The model used here allows fractional values:

$$x_j \in \mathbb{R}_+$$

Where $\mathbb{R}_+$ denotes the set of non-negative real numbers (positive and zero). So, for example $x_1 = 125.7$ units of D_1 is mathematically allowed. For actual indivisible electronic devices this may look strange, but this example is intentionally treated as a **continuous model** to introduce *Linear Programming*. Later, integer restrictions could instead be imposed if needed.

Step 7: Complete model. Putting everything together, the complete mathematical optimization model is:

$$\boxed{\begin{aligned} \max \quad & 1.6x_1 + x_2 + 2x_3 \\ \text{s.t.} \quad & 80x_1 + 70x_2 + 120x_3 \leq 30000 \\ & 70x_1 + 90x_2 + 20x_3 \leq 25000 \\ & 40x_1 + 30x_2 + 20x_3 \leq 18000 \\ & x_1, x_2, x_3 \geq 0. \end{aligned}}$$

Everything here is **linear**:

- Objective function: linear;
- Constraints: linear;
- Variables: continuous.

Therefore this is a Linear Programming problem (LP). We will formally define LP later.

Deepening: How to read a constraint

We should develop the ability to look at:

$$80x_1 + 70x_2 + 120x_3 \leq 30000$$

And immediately interpret it verbally: "*the total assembly time consumed by the production plan cannot exceed the available assembly time*". Likewise:

$$70x_1 + 90x_2 + 20x_3 \leq 25000$$

Means: "*the total refinement time cannot exceed the refinement capacity*". This translation between mathematics and natural language is essential for modeling questions in the **exam**.

Example 9: Portfolio selection (binary decision variables)

The second example is structurally different. An insurance company has eight possible investments:

Investment	Sector/Type	Country
A	Automotive	Germany
B	Automotive	Italy
C	ICT	USA
D	ICT	Italy
E	Real estate	Italy
F	Real estate	France
G	Short-term treasury bonds	Italy
H	Long-term treasury bonds	UK

Note that ICT stands for *Information and Communication Technology*. It refers broadly to the technology sector: computing, software, telecommunications, electronics, IT services, etc. Instead, *treasury bond* is a government-issued debt security. Very roughly, when we buy one, we are lending money to a government, and the government promises to repay us according to specified terms, generally with interest.

Each investment has:

- A required capital c_j;
- An expected return r_j;

- A geographical location;
- A sector.

The available capital is:

$$600 \text{ k€}$$

There are also diversification rules:

- Select at most 5 investments;
- Select at most 3 investments in Italy;
- Select at most 3 investments abroad.

The goal is: select investments maximizing expected return.

Why the variable type is different. In production planning (previous example), asking:

$$x_j = \text{how much of product } j?$$

Made sense. Here the decision is: "*do we select investment j or not?*". There are **only two possibilities**. So define:

$$\boxed{x_j = \begin{cases} 1 & \text{if investment } j \text{ is selected,} \\ 0 & \text{otherwise.} \end{cases}}$$

Hence:

$$x_j \in \{0, 1\}$$

These are **binary variables**. This modeling trick is fundamental. Whenever the decision is of the form:

$$\text{yes/no}$$

$$\text{open/closed}$$

$$\text{selected/not selected}$$

$$\text{build/do not build}$$

Binary variables are a **natural representation**.

Objective function. Investment j requires capital c_j and provides expected return rate r_j. Its expected monetary return is:

$$c_j r_j$$

If it is **selected**:

$$x_j = 1$$

So its contribution is:

$$c_j r_j$$

If it is **not selected**:

$$x_j = 0$$

So:

$$c_j r_j x_j = 0$$

Therefore **total expected return** is:

$$\sum_{j=1}^{8} c_j r_j x_j$$

Where 8 is the number of investments. And the objective becomes:

$$\boxed{\max \sum_{j=1}^{8} c_j r_j x_j}$$

This illustrates another extremely common modeling device:

$$\text{coefficient} \times \text{binary variable}$$

Automatically activates or deactivates a contribution.

Capital constraint. If investment j is selected, it consumes c_j units of capital. Therefore total invested capital is:

$$\sum_{j=1}^{8} c_j x_j$$

Available capital is 600 k€, so:

$$\boxed{\sum_{j=1}^{8} c_j x_j \leq 600}$$

Maximum number of investments. Because $x_j = 1$ exactly when investment j is selected:

$$\sum_{j=1}^{8} x_j$$

Counts the number of selected investments. Therefore:

$$\boxed{\sum_{j=1}^{8} x_j \leq 5}$$

Means: *select at most five investments.* This is an incredibly useful property of binary variables:

$$\sum_{j} x_j = \text{number of selected items} \tag{1}$$

When every $x_j \in \{0, 1\}$.

Geographic diversification. The **investments** located in **Italy** are:

$$B, D, E, G$$

Their variables are:

$$x_2, x_4, x_5, x_7$$

At most three may be selected:

$$\boxed{x_2 + x_4 + x_5 + x_7 \leq 3}$$

Similarly, the **foreign investments** are:

$$A, C, F, H$$

So:

$$\boxed{x_1 + x_3 + x_6 + x_8 \leq 3}$$

Again, the sum simply **counts selected investments belonging to a category**. This is a basic modeling pattern worth memorizing:

$$\sum_{j \in S} x_j \leq k \tag{2}$$

Means: "*select at most k elements from set S*". Likewise:

$$\sum_{j \in S} x_j \geq k$$

Means: "*select at least k elements from set S*". And:

$$\sum_{j \in S} x_j = k$$

Means: "*select exactly k elements from set S*".

Complete portfolio model. Putting everything together, the complete mathematical optimization model is:

$$\boxed{\begin{aligned} \max \quad & \sum_{j=1}^{8} c_j r_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^{8} c_j x_j \leq 600 \\ & \sum_{j=1}^{8} x_j \leq 5 \\ & x_2 + x_4 + x_5 + x_7 \leq 3 \\ & x_1 + x_3 + x_6 + x_8 \leq 3 \\ & x_j \in \{0, 1\}, \qquad j = 1, \ldots, 8 \end{aligned}}$$

The **objective** and **constraints** are still **linear**. But the **variables** are **not continuous**:

$$x_j \in \{0, 1\}$$

Therefore this is an: Integer Linear Programming problem (ILP), or more precisely, a **binary linear optimization problem**.

≠ LP vs ILP: first important distinction

The two examples are intentionally placed next to each other.

- **Production planning**

  $$x_j \geq 0, \qquad x_j \in \mathbb{R}$$

  The variables can vary continuously. Therefore it is a **Linear Programming problem (LP)**.

- **Portfolio selection**

  $$x_j \in \{0, 1\}$$

  The variables are discrete. Therefore it is an **Integer Linear Programming problem (ILP)**.

Even though both models use linear equations and inequalities, the **variable domains are different**, and that difference matters enormously algorithmically. We will later see that integer optimization is generally much harder than ordinary linear programming.

Example 10: Modeling logical implications with constraints

We continue with the **portfolio selection problem** (page 27), but now add a logical rule: if at least one ICT investment is selected, then at least one treasury bond must also be selected.

From the table on page 27, we know that the ICT investments are:

$$C, D$$

With variables:

$$x_3, x_4$$

The treasury bonds are:

$$G, H$$

With variables:

$$x_7, x_8$$

Recall that all variables are binary:

$$x_j \in \{0, 1\}$$

So we want to **translate the logical statement**:

$$(x_3 = 1 \text{ or } x_4 = 1) \Longrightarrow (x_7 = 1 \text{ or } x_8 = 1)$$

Into a linear constraint.

Counting selected investments. Because the variables are binary:

$$x_3 + x_4$$

Counts the number of selected ICT investments. Therefore:

$$x_3 + x_4 = \begin{cases} 0 & \text{no ICT investment} \\ 1 & \text{one ICT investment} \\ 2 & \text{both ICT investments} \end{cases}$$

Similarly:

$$x_7 + x_8$$

Counts selected treasury bonds. We need:

$$x_3 + x_4 > 0 \quad \Longrightarrow \quad x_7 + x_8 \geq 1.$$

The model uses:

$$\boxed{\frac{x_3 + x_4}{2} \leq x_7 + x_8}$$

Let's understand **why this works**.

No ICT investment. Suppose:

$$x_3 = x_4 = 0$$

Then:

$$\frac{x_3 + x_4}{2} = 0$$

The constraint becomes:

$$0 \leq x_7 + x_8$$

This is always true. Therefore, if we select no ICT investment, there is **no obligation** to select a treasury bond. Exactly what we want.

One ICT investment. Suppose:

$$x_3 + x_4 = 1$$

Then:

$$\frac{x_3 + x_4}{2} = \frac{1}{2}$$

The constraint becomes:

$$\frac{1}{2} \leq x_7 + x_8$$

But $x_7 + x_8$ is integer because x_7, x_8 are binary. Its possible values are:

$$0, 1, 2$$

Therefore:

$$x_7 + x_8 \geq \frac{1}{2}$$

Actually forces:

$$x_7 + x_8 \geq 1$$

Because $x_7 + x_8$ cannot be 0 and the next possible value is 1. So **at least one treasury bond must be selected**.

Both ICT investments. Suppose:

$$x_3 = x_4 = 1$$

Then:

$$\frac{x_3 + x_4}{2} = 1$$

Therefore:

$$1 \leq x_7 + x_8$$

Again **at least one treasury bond** must be selected. Notice that selecting both ICT investments does **not** require two treasury bonds. The logical rule only says: "*if **any** ICT investment is present, select **at least once** bond*". So the formulation is correct.

Equivalent form. We can multiply both sides by 2:

$$x_3 + x_4 \leq 2(x_7 + x_8)$$

Which may make the logic easier to see.

A general modeling pattern for *Logical Implications*

From the previous example, we can extract a **general modeling pattern**. Suppose S is a group containing m binary variables, and we want: if at least one element of S is selected, then at least one element of T must be selected. A formulation is:

$$\frac{1}{m} \sum_{j \in S} x_j \leq \sum_{k \in T} x_k \tag{3}$$

Or equivalently:

$$\sum_{j \in S} x_j \leq m \sum_{k \in T} x_k \tag{4}$$

This kind of construction will become very important in **Integer Linear Programming (ILP)**, because binary variables allow us to encode logical decisions using algebra.

Example 11: Facility location (nonlinear models)

The third example is geometrically very different. There are three oil pits:

$$A = (0, 0)$$

$$B = (300, 0)$$

$$C = (240, 300)$$

We must choose **where to build a refinery**, which will be connected to the pits by pipelines (i.e., straight lines). There is also a point:

$$D = (100, 200)$$

And the refinery must be at least 100 km away from D.

The oil pits will be connected directly to the refinery by pipelines. The important detail is that the pipeline cost is proportional to the **square of its length**. We want to determine the refinery location minimizing total pipeline cost.

Deepening: What does "*cost proportional to the square of the length*" mean?

Suppose a pipeline has length L. The exercise assumes its cost behaves like:

$$\text{cost} = kL^2$$

For some positive constant k. So if one pipeline has length (i.e., distance from one oil pit to the refinery):

$$L = 10$$

Its cost is proportional to:

$$10^2 = 100$$

If another has length:

$$L = 20$$

Its cost is proportional to:

$$20^2 = 400$$

So doubling the length does not merely double the cost:

$$20 = 2 \cdot 10$$

But:

$$20^2 = 4 \cdot 10^2$$

Hence the cost becomes four times as large. The exercise gives this as a modeling assumption. We do not need to justify physically why pipeline construction behaves exactly like L^2; we simply model what the problem states.

Step 1: Decision variables. *What do we decide?* The refinery

coordinates. So we define two variables:

$$x_1 = \text{x-coordinate of the refinery}$$

$$x_2 = \text{y-coordinate of the refinery}$$

Therefore:

$$(x_1, x_2) \in \mathbb{R}^2$$

Unlike the portfolio problem, these are **continuous variables** (i.e., they can take any real value).

📏 **Step 2: Distance to each oil pit.** Recall the Euclidean distance between points:

$$(x_1, x_2)$$

And:

$$(a, b)$$

Is:

$$d = \sqrt{(x_1 - a)^2 + (x_2 - b)^2}$$

But pipeline cost is proportional to the **square** of the distance. Therefore:

$$d^2 = \left(\sqrt{(x_1 - a)^2 + (x_2 - b)^2}\right)^2 = (x_1 - a)^2 + (x_2 - b)^2$$

This is convenient because the square root disappears.

- **Pipeline from** A. Since $A = (0, 0)$, the squared length is:

  $$(x_1 - 0)^2 + (x_2 - 0)^2$$

  Thus:

  $$x_1^2 + x_2^2$$

- **Pipeline from** B. Since $B = (300, 0)$, the squared length is:

  $$(x_1 - 300)^2 + x_2^2$$

- **Pipeline from** C. Since $C = (240, 300)$, the squared length is:

  $$(x_1 - 240)^2 + (x_2 - 300)^2$$

🎯 **Step 3: Objective function.** Total cost is proportional to the sum of these squared distances. Therefore:

$$\begin{aligned} \min z = \ & \left[x_1^2 + x_2^2\right] \\ & + \left[(x_1 - 300)^2 + x_2^2\right] \\ & + \left[(x_1 - 240)^2 + (x_2 - 300)^2\right] \end{aligned}$$

This already differs fundamentally from our previous models. For production planning, the objective looked like:

$$1.6x_1 + x_2 + 2x_3$$

It was **linear**. Here we have terms such as x_1^2 and x_2^2. Therefore the objective is **nonlinear**. More precisely, it is a **quadratic function**.

Step 4: Forbidden area around D. The refinery must remain at least 100 km away from D:

$$D = (100, 200)$$

The distance between the refinery and D is:

$$\sqrt{(x_1 - 100)^2 + (x_2 - 200)^2}$$

Therefore the requirement becomes:

$$\boxed{\sqrt{(x_1 - 100)^2 + (x_2 - 200)^2} \geq 100}$$

To enhance geometric intuition, since both sides are nonnegative, we can square both sides:

$$(x_1 - 100)^2 + (x_2 - 200)^2 \geq 10000$$

This is a **nonlinear constraint** because it contains squared decision variables. Geometrically, the equation:

$$\sqrt{(x_1 - 100)^2 + (x_2 - 200)^2} = 100$$

Describes a circle centered at $(100, 200)$ with radius 100. The inequality ≥ 100 means the refinery must lie ***on* or *outside* that circle**. It cannot lie inside the forbidden disk.

Complete facility-location model. Putting everything together, we have:

$$\boxed{\begin{aligned} \min \quad & x_1^2 + x_2^2 \\ & + (x_1 - 300)^2 + x_2^2 \\ & + (x_1 - 240)^2 + (x_2 - 300)^2 \\ \text{s.t.} \quad & (x_1 - 100)^2 + (x_2 - 200)^2 \geq 10000 \\ & x_1, x_2 \in \mathbb{R} \end{aligned}}$$

This is a Nonlinear Programming problem (NLP) because the objective and constraints are not all linear.

Why this is different from LP. A linear function can contain:

$$3x_1 + 7x_2 - 2x_3$$

But it cannot contain: x_1^2, x_1x_2, $\sqrt{x_1}$, e^{x_1}, etc. So LP variables appear only linearly, whereas here:

$$(x_1 - 300)^2$$

Contains squared decision variables. Hence it is a nonlinear model.

Key Takeaways 4

The most important modeling template is:

Decision variables :	What do I control?
Objective :	What do I optimize?
Constraints :	What must be respected?
Domains :	What values can the variables take?

For **resource constraints**:

$$\boxed{\text{total consumption} \leq \text{available capacity}}$$

For **binary variables**:

$$x_j = \begin{cases} 1 & \text{selected} \\ 0 & \text{not selected} \end{cases}$$

And therefore:

$$\boxed{\sum_j x_j = \text{number of selected items}}$$

Finally:

	Production planning	Portfolio selection
Variables	continuous	binary
Objective	linear	linear
Constraints	linear	linear
Class	LP	ILP

For **binary modeling**, remember that logical statements can often be encoded through inequalities. In the previous examples:

$$\text{ICT selected} \Rightarrow \text{treasury bond selected}$$

Mathematically, this was expressed as:

$$x_3 + x_4 \leq 2(x_7 + x_8)$$

For geometric modeling, remember that the Euclidean distance is:

$$d((x_1, x_2), (a, b)) = \sqrt{(x_1 - a)^2 + (x_2 - b)^2}$$

And squared-distance costs naturally produce **quadratic objectives**. Most importantly, this section has now introduced the four fundamental pieces we will repeatedly use:

$$\text{variables} + \text{objective} + \text{constraints} + \text{domains}$$

References

[1] Braz Pascoal Marta Margarida. Foundations of Operations Research. Slides from the HPC-E master's degree course on Politecnico di Milano, 2026.

Index

C

Clique 13

D

Decision-Making Problem 9

M

Maximum Clique Problem 13
Multi-Objective Problem 13
Multicriteria Problem 13

O

Operations Research (OR) 5